# Tarea de laboratorio 4 – Listas

**Objetivos**

- Practicar en Java la utilización de las listas como estructura de datos, aplicando operaciones de filtrado, ordenación y agrupación para organizar información.

**Descripción de la práctica:** Sistema de listas electorales

Con motivo de las elecciones, la Junta Electoral requiere un programa en Java para procesar, comprobar la paridad y organizar las candidaturas de las formaciones políticas (BB y BSOE) en las circunscripciones de Madrid y Barcelona, calculando la asignación de candidatos electos según los votos obtenidos.

El programa leerá la información de las candidaturas desde el archivo CSV `candidaturas.csv` en el cual cada línea representa la siguiente información de cada candidato:

- *Partido*: Nombre de la formación política (BB o BSOE).
- *Circunscripción*: Nombre del distrito electoral (Madrid o Barcelona).
- *Posición*: Orden numérico del candidato en la lista cerrada del partido (1, 2, 3…).
- *Nombre del Candidato/a*: Nombre y apellidos del candidato/a.
- *Género*: Representación del género mediante H (Hombre) o M (Mujer).

El programa deberá realizar las siguientes tareas:

1. **Lectura:** Leer la información del archivo `candidaturas.csv` e instanciar los objetos necesarios para representar los candidatos/as.
2. **Generación de listas electorales:** A partir de la información leída se generará la lista electoral de cada partido en cada circunscripción. El programa mostrará la lista ordenada[1] de candidatos/as de cada partido en cada circunscripción conforme a su posición.
3. **Verificación de paridad:** Se comprobará la paridad de cada lista electoral. Una lista cumple el criterio de paridad si cada género tiene una representación de entre el 40% y el 60% entre sus integrantes. Para aquellas listas electorales que no cumplan el criterio de paridad el programa mostrará un mensaje de alerta.
4. **Obtención de votos:** El programa pedirá al usuario que introduzca la cantidad de votos obtenida por cada partido en cada circunscripción. Estos valores serán introducidos por el usuario mediante el teclado.
5. **Asignación de escaños:** Cada partido obtiene 1 escaño por cada corte completo de 30.000 votos. El programa mostrará la cantidad de escaños obtenida por cada partido en cada circunscripción.
6. **Asignación de candidatos electos:** Para cada partido en cada circunscripción, los candidatos/as electos/as serán los *k* primeros de su lista ordenada, correspondiendo *k* a la cantidad de escaños obtenida. El programa mostrará la lista ordenada de candidatos/as electos/as para cada partido en cada circunscripción.

Se proporciona una base de código del programa parcialmente implementada, la cual consiste en:

- *Genero*: enumeración que representa el género de los candidatos: `public enum Genero { H, M }`.
- *Candidato:* clase que representa información de los candidatos. Se crean instancias de esta clase a partir de la información leída del fichero CSV.

[1] Para la implementación de la ordenación, se recomienda utilizar la función `Collection.sort()` de Java. Nótese que un objeto `List` *es un* objeto `Collection`.

- *ListaElectoral:* clase que representa información de la lista electoral de un partido en una circunscripción, con su lista de candidatos/as y que permite determinar la lista de electos/as en función de los votos y escaños obtenidos.
- *GestiónElectoral:* clase principal de sistema que permite realizar la gestión de listas electorales y la votación de acuerdo a las tareas indicadas anteriormente.

Se requiere completar la implementación del sistema de acuerdo a lo indicado arriba, para lo que se requiere implementar los siguientes métodos en las siguientes 2 clases:

- ListaElectoral
  - agregarCandidato
  - cumpleParidad
  - calcularEscanos
  - getCandidatosElectos

- GestionElectoral
  - generarListasElectorales
  - verificarParidad
  - mostrarAlertasParidad
  - mostrarListaElectos

**Requisitos técnicos**

- Se deben utilizar las implementaciones adecuadas de listas de la librería estándar de Java.
- Se proporciona en el Campus Virtual el fichero de candidatos, así como el código base mencionado que se debe completar.
- Los programas no podrán contener moldes en relación con las listas ni ninguna referencia a un `Object`.
- Para los métodos que se deben completar se deberá realizar su documentación interna. En Campus Virtual se puede encontrar información para generar esta documentación (sección Laboratorio).
- Con el fin de utilizar sistemas antiplagio y de detección semántica de similitudes, los ficheros de código fuente no podrán contener datos sobre los estudiantes de carácter personal como el nombre, el correo electrónico… En su lugar, se identificará a los autores y el grupo de prácticas en los comentarios del código con las iniciales de los estudiantes y el código de grupo asignado.
- No se podrán utilizar caracteres especiales en los nombres de los ficheros.

**Normas de realización**

- La práctica deberá realizarla cada uno de los equipos de trabajo formados previamente en clase de laboratorio. No podrá realizarse de forma individual.
- Por motivos pedagógicos, no estará permitido utilizar asistentes inteligentes de generación de código (programas), como Chat-GPT, GitHub Copilot o Claude, y se penalizarán aquellas prácticas que se alejen de la filosofía de la programación estructurada y la programación orientada a objetos.